



INSTITUTO DE CIÊNCIA E TECNOLOGIA
UNIVERSIDADE ESTADUAL PAULISTA

RELATÓRIO DO PROJETO DE ESTRUTURA DE DADOS

SISTEMA DE ANÁLISE DE DADOS DA NBA COM
ESTRUTURAS DE DADOS

Professor:

Dr. Leopoldo André Dutra
Lusquino Filho

Alunos - RA :

Pedro Ribeiro Silva Coelho -
251270467

PEDRO RIBEIRO SILVA COELHO

Sistema de Análise de Dados da NBA com Estruturas de Dados
PROJETO DE ESTRUTURA DE DADOS

SOROCABA
2026

Conteúdo

1	Introdução	2
2	Objetivos	2
3	Domínio e Dataset	2
4	Tratamento de Dados	3
5	Estruturas de Dados Implementadas	3
6	Operações Adicionais de Análise	5
7	Análise de Desempenho e Metodologia de Simulação	6
7.1	Benchmark das Estruturas	6
7.2	Otimização do Benchmark	9
8	Benchmarks com Restrições - Tabela Hash e Lista	11
8.1	R5 - Descarte Automático de Dados Antigos Quando a Memória Atinge um Limite Crítico.	11
8.2	R7 - Restrição de Orçamento Computacional	11
8.3	R12 - Simulação de Alta Latência no Acesso a Banco de Dados	12
8.4	R18 - Dados	13
8.5	R24 - Uso de Algoritmos Menos Otimizados	14
9	Complexidade Temporal Teórica e Assintótica Real	15
9.1	Lista Encadeada Simples	15
9.2	Árvore AVL	16
9.3	Tabela Hash	16
9.4	Skip List	17
9.5	Bloom Filter	17
10	Conclusão	19
11	Uso de Inteligência Artificial	19

1 Introdução

O desenvolvimento de um software moderno exige frequentemente a manipulação eficiente de grandes volumes de dados em ambientes dinâmicos e de constante atualização. Sistemas críticos dependem da organização coerente para que as informações possam ser inseridas, atualizadas e recuperadas de maneira contínua, evidenciando que sistemas reais dificilmente se amparam em uma única estrutura de dados isolada. Dessa forma, o presente trabalho descreve um sistema computacional de tempo real desenvolvido em linguagem C para o gerenciamento analítico de partidas da NBA. O principal objetivo não é apenas apresentar um software funcional, mas demonstrar de forma empírica e fundamentada o domínio conceitual sobre diferentes estratégias de armazenamento de dados e avaliar o desempenho comparativo entre cada estrutura de dados distintas sob condições intensas de dados.

2 Objetivos

Os objetivos do projeto são:

1. Utilizar 5 estruturas de dados, sendo três da ementa do curso, e duas fora da ementa;
2. Realizar operações de adicionar, remover e procurar por elementos, além de efetuar 3 operações adicionais;
3. Efetuar benchmarks, com e sem restrições, e comparar o desempenho de estruturas de dados que resolvem os mesmos tipos de problemas;
4. Fazer, em pelo menos uma estrutura, um tipo de otimização, e compará-la com sua versão não-otimizada.

3 Domínio e Dataset

Para fundamentar as análises experimentais, foi selecionado o domínio de estatísticas da National Basketball Association (NBA), utilizando o repositório público "NBA-Data-2010-2024", encontrado no site "GitHub", feito pelo autor Vitalii Korolyk. O dataset mapeia as estatísticas fundamentais através de uma estrutura de Registro, capaz de concentrar o desempenho individual de cada atleta por partida, armazenando variáveis como pontos, minutos, rebotes, assistências, roubos de bola e tentativas de lances livres. Este dataset foi escolhido com o objetivo de obter um grande volumes de dados reais e contínuos, e que abre margem à exploração da inserção e uso de diversas estruturas de dados. O trabalho obedece estritamente ao requisito de manipulação massiva (acima de 10.000 amostras) para garantir que as diferenças de tempo de execução e complexidade espacial entre os algoritmos possam ser validadas e contrastadas visivelmente no benchmark. Para isso, esse dataset selecionado conta com mais de 600.000 registros.

4 Tratamento de Dados

A base de dados selecionada está em formato .csv, separado por vírgulas (.). Vale-se ressaltar que os minutos jogados por cada jogador vem no formato "MM:SS", que ao ser lido como um número pelo código, ocasionaria num erro. Para tratar esse erro, foi feita uma função "parse-minutos" que separa a parte antes dos dois-pontos, assim, é possível obter o tempo de minutos em quadra de cada jogador num jogo específico. A fim de tornar essa variável mais simples, os segundos foram desconsiderados.

Além disso, deve-se ressaltar que não existe coluna de vitória/derrota neste arquivo, pois como o arquivo é por jogador, ele não informa as vitórias do time. Com isso, para obter a informação se nesse jogo em específico o jogador venceu ou perdeu, foi necessário calcular a soma de pontos de todos os jogadores que jogaram aquela partida para obter esse dado.

Ademais, existem linhas "DNP" (Did Not Play). Jogadores que ficaram no banco aparecem no CSV com os minutos em branco e o comment dizendo "DNP". Dessa forma, jogadores sem minutos em quadra não possuem um desempenho real, logo não são considerados pelo sistema.

E por fim, para calcular o aproveitamento de lances livres, deve-se evitar divisões por zero. Esse erro foi tratado e contornado no código, ao ser calculado o aproveitamento de jogadores que jogaram uma partida mas não realizaram nenhum lance livre.

5 Estruturas de Dados Implementadas

Visando atender à arquitetura conceitual comum exigida no escopo do projeto, o sistema integra e compara cinco estruturas distintas de gerenciamento em memória, divididas entre modelos clássicos e probabilísticos:

Lista Encadeada Simples: Modelo clássico implementado para servir como base (baseline) de comparação. Embora ofereça inserção em complexidade $O(1)$ no início, sua busca $O(n)$ exige o percurso nó a nó. Uma variação, chamada Lista Compactada, também foi explorada no código para alocar blocos contínuos, reduzindo chamadas dinâmicas e beneficiando a leitura de cache do processador. Neste projeto, a Lista Encadeada Simples tem a função de inserir e/ou remover jogadores, a partir do "id" do jogador, seu nome e sua quantidade de pontos.

Árvore AVL: Estrutura hierárquica clássica binária balanceada baseada no cálculo da altura de seus sub-nós. Através da aplicação matemática de rotações à direita e à esquerda nos momentos de inserção e remoção, ela garante que as operações essenciais se mantenham restritas à complexidade logarítmica ($O(\log n)$), evitando a degeneração dos acessos. Neste projeto, a Árvore AVL realiza buscas de jogadores a partir de seu "id", e diz se o "id" do atleta foi ou não encontrado.

Tabela Hash: Estrutura focada em indexação direta. No projeto, as colisões são tratadas usando encadeamento separado, onde cada "balde" (bucket) gerencia sua própria sublista de registros. Possui mecanismos de monitoramento para fator de carga e extensão da maior cadeia, permitindo análises empíricas precisas sobre a distribuição e o agrupamento uniforme dos dados. Neste projeto a Tabela Hash foi usada para encontrar jogadores através de seu "id", e diz se foi ou não encontrado.

Skip List (Estrutura Fora da Ementa): Uma robusta estrutura probabilística pautada por múltiplos níveis progressivos de navegação expressa. Oferece vantagens substanciais na simplicidade de implementação mantendo uma eficiência probabilística de $O(\log n)$ para buscas rápidas sobre dados ordenados, o que a torna uma adversária direta no benchmark contra a Árvore AVL. Neste projeto a Skip List é usada para buscar e remover jogadores.

Bloom Filter (Estrutura Fora da Ementa): Solução probabilística concebida para atuar no controle de pertinência. Não guarda os dados físicos, mas opera ligando e validando um mapa de vetores de bits operado por múltiplas chaves hashes. Esta escolha reflete uma preocupação iminente no uso contido de memória para consultas iniciais mais velozes e aceitação tolerável de "falsos positivos" em larga escala, servindo frequentemente de barreira primária otimizada. Neste projeto, o Bloom Filter é usado para verificar se um jogador foi ou não inserido no sistema através de uma varredura probabilística. A respeito do benchmark do Bloom Filter, que será trabalhado mais adiante nesse relatório, não é possível fazer com este, haja vista que não é possível fazer a operação de remoção, somente inserção e busca.

6 Operações Adicionais de Análise

O tratamento analítico transcende o simples arquivamento e compreende três fluxos operacionais adicionais sobre as métricas da NBA:

Cálculo Estatístico Extenso (Operação 1): Processa uma varredura agregando estatísticas detalhadas do jogador por temporada. Retorna variáveis complexas, como médias aritméticas globais e cálculo de desvio padrão.

Classificação Baseada nas Estruturas (Operação 2 - Ranking de MVP da temporada): Determina um índice composto de eficiência em quadra e pontuação para elencar de modo decrescente os desempenhos de temporada, fornecendo uma base sólida de ordenação. Essa operação utiliza todas as estatísticas dos jogadores para fundamentar e validar esse ranking, já que se trata de um ranking sobre eficiência em quadra, não considerando somente uma das habilidades do atleta.

Filtragem e Agrupamento (Operação 3 - Vitórias): Processamento cruzado onde o agrupamento por identificadores únicos de partidas revela o consolidado de vitórias acumuladas por cada franquia e temporada, demonstrando filtragem analítica refinada dos logs da liga.

7 Análise de Desempenho e Metodologia de Simulação

Para a aprovação real do software, uma bateria de testes sistemáticos ("benchmark") foi construída isoladamente em módulos nativos de compilação. Estes executáveis apartados conduzem medições que aferem a discrepância entre o desempenho teórico descrito na literatura de algoritmos e o comportamento empírico verificado em simulação sob restrições físicas de arquitetura. É esperado que as otimizações focadas no uso coerente da hierarquia de memória mostrem vantagem contundente nas iterações e no acesso imediato, consolidando o entendimento de que uma decisão de projeto em sistemas de grandes volumes sempre dependerá da análise criteriosa do trade-off (compromisso) entre CPU e escalabilidade do armazenamento.

7.1 Benchmark das Estruturas

A análise dos dados experimentais revela comportamentos distintos entre as estruturas de dados avaliadas (Lista, AVL, Hash e Skiplist) à medida que o volume de dados (N) cresce de 1.000 para 200.000. Os resultados evidenciam a otimização do tempo de execução e o consumo de memória de cada uma das estruturas. A Tabela 1 apresenta os dados obtidos a partir dos testes de INserção, Busca, Remoção e Memória de cada umas das estruturas. O Bloom Filter não entra na corrida de Benchmark, porque ele responde uma pergunta diferente — ele não recupera o registro, só diz se ele provavelmente existe.

A respeito do Tempo de Execução (Operações), a Lista Encadeada apresentou o pior desempenho em operações de busca e remoção, evidenciando uma complexidade de tempo linear, $O(N)$. Para $N = 1000$, a busca custava 2.32 us/op, mas saltou para absurdos 7736.46 us/op em $N = 200000$. No entanto, o tempo de inserção manteve-se praticamente constante e muito eficiente (cerca de 0.11 us/op), sugerindo que a inserção ocorre no início ou no fim da estrutura. A Árvore AVL e a Skiplist demonstraram alta eficiência e escalabilidade. Os tempos de busca e remoção cresceram a taxas muito lentas, características de complexidade logarítmica, $O(\log N)$. A AVL mostrou-se ligeiramente superior à Skiplist na maioria das operações de busca e remoção para grandes volumes de dados. O tempo de inserção na AVL, no entanto, sofreu um leve aumento no maior volume de dados (1.27 us/op em $N = 200.000$), devido ao custo das rotações para manter o balanceamento da árvore. A Tabela Hash foi a estrutura mais rápida de forma isolada para quase todas as operações, aproximando-se de uma complexidade constante, $O(1)$. Em $N = 200.000$, a busca custou apenas 0.19 us/op. É fundamental observar, no entanto, que o experimento registrou colisões=0 e uma carga de 1.00, o que representa um cenário ideal de dispersão e dimensionamento prévio perfeito da tabela.

Sobre o Consumo de Memória, a eficiência de tempo obtida pelas estruturas mais rápidas tem um custo claro em espaço; a Lista Encadeada foi a estrutura mais econômica em todos os cenários. Em $N = 200.000$, consumiu 25000.0 KB. Isso ocorre porque ela armazena apenas o dado e o ponteiro para o próximo nó, sem a necessidade de manter nós

de balanceamento ou alocações vazias. A Árvore AVL, Hash e Skiplist consumiram significativamente mais memória. Em $N = 200.000$, a AVL consumiu o maior valor absoluto (28125.0 KB), seguida pela Skiplist (28093.7 KB) e pela Tabela Hash (26562.5 KB). Na AVL, esse custo extra se deve ao armazenamento de ponteiros adicionais (esquerda, direita) e fatores de balanceamento por nó. Na Skiplist, o custo é gerado pelos múltiplos ponteiros nos vários níveis probabilísticos. Na Hash, o alto consumo reflete a alocação de um vetor suficientemente grande para manter o fator de carga em 1.00 sem colisões.

Dessa forma, pode-se concluir que a escolha da estrutura ideal depende do caso de uso da aplicação. A Tabela Hash é a escolha definitiva quando a velocidade de acesso e modificação é o fator crítico e há memória abundante para garantir um baixo índice de colisões. A Árvore AVL provou ser a melhor estrutura de uso geral para cenários onde a garantia do tempo logarítmico no pior caso é necessária, especialmente se os dados precisarem ser mantidos de forma ordenada (algo que a Hash não oferece). A Skiplist apresentou-se como uma excelente alternativa probabilística à AVL, com desempenho e uso de memória bastante similares. A Lista deve ser evitada para grandes volumes de dados caso haja necessidade de buscas ou remoções frequentes, sendo viável apenas para cenários de tamanho reduzido ou fluxos de dados de apenas inserção e leitura sequencial.

Logo abaixo, pode-se observar a tabela de desempenho de cada estrutura e os gráficos de cada operação: Busca, Inserção, Remoção e da Memória, em função do número de dados inseridos em cada estrutura. A partir desses gráficos, evidencia-se o funcionamento e validação da complexidade algorítmica de cada uma das estruturas como discutido anteriormente.

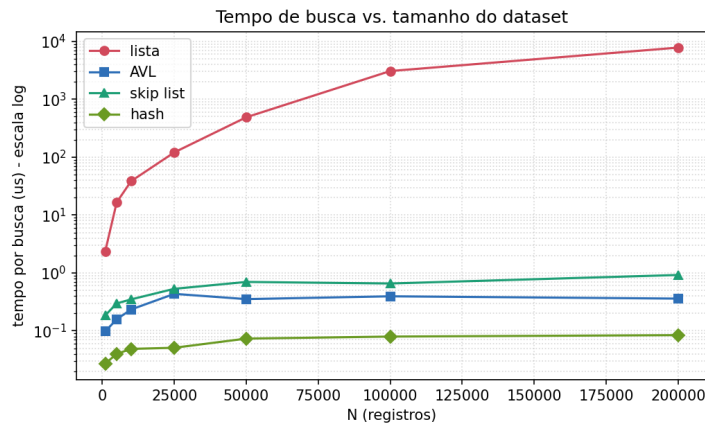


Figura 1: Gráfico do tempo de busca em função do número de dados.

Tabela 1: Desempenho das Estruturas de Dados

Estrutura	N	Inserção (us/op)	Busca (us/op)	Remoção (us/op)	Memória (KB)	Observações
Lista	1000	0.2773	2.3274	1.0555	125.0	colisoes=0, carga=1.00
Avl	1000	0.3904	0.1054	0.1006	140.6	
Hash	1000	0.1305	0.0332	0.0299	132.8	
Skiplist	1000	0.5827	0.2215	0.0687	125.0	
Lista	5000	0.1240	19.0456	34.2095	625.0	colisoes=0, carga=1.00
Avl	5000	0.3943	0.1783	0.4610	703.1	
Hash	5000	0.1107	0.0463	0.0707	664.1	
Skiplist	5000	0.3413	0.3467	0.1894	671.2	
Lista	10000	0.1092	40.2839	71.0551	1250.0	colisoes=0, carga=1.00
Avl	10000	0.3840	0.2370	0.2535	1406.2	
Hash	10000	0.1020	0.0510	0.0657	1328.1	
Skiplist	10000	0.3271	0.3570	0.0961	1375.5	
Lista	25000	0.1144	145.7272	257.3553	3125.0	colisoes=0, carga=1.00
Avl	25000	0.4833	0.2909	0.2738	3515.6	
Hash	25000	0.1146	0.0574	0.0579	3320.3	
Skiplist	25000	0.3659	0.6714	0.4067	3482.0	
Lista	50000	0.1196	600.4697	1218.6407	6250.0	colisoes=0, carga=1.00
Avl	50000	0.4839	0.5150	0.2899	7031.2	
Hash	50000	0.1173	0.0817	0.0602	6640.6	
Skiplist	50000	0.3496	0.7477	0.1233	7002.7	
Lista	100000	0.0992	2992.6612	3260.2976	12500.0	colisoes=0, carga=1.00
Avl	100000	0.5422	0.3811	0.2768	14062.5	
Hash	100000	0.1284	0.0970	0.0712	13281.2	
Skiplist	100000	0.3910	0.7318	0.1495	14035.1	
Lista	200000	0.1108	7736.4681	9635.0133	25000.0	colisoes=0, carga=1.00
Avl	200000	1.2767	0.6937	0.6092	28125.0	
Hash	200000	0.2666	0.1977	0.1585	26562.5	
Skiplist	200000	0.9900	1.8211	0.4173	28093.7	

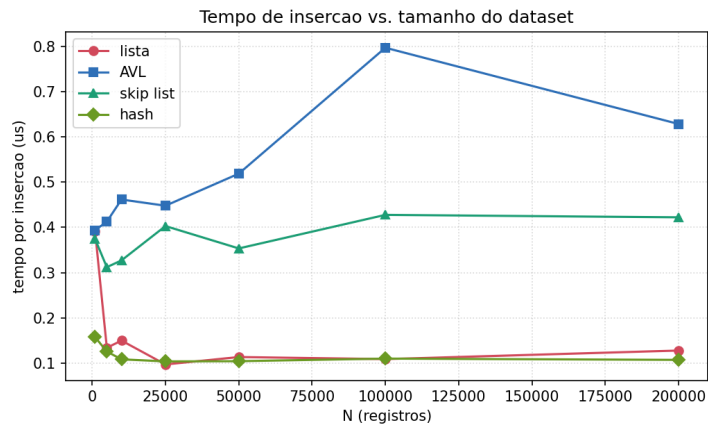


Figura 2: Gráfico do tempo de inserção em função do número de dados.

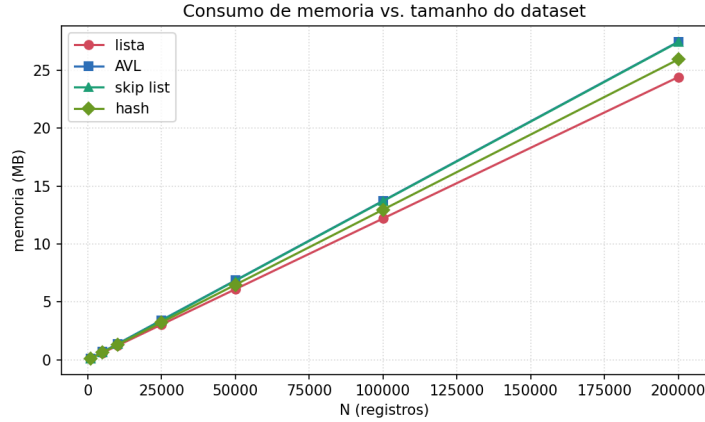


Figura 3: Gráfico do tempo do consumo de memória em função do número de dados.

7.2 Otimização do Benchmark

A estrutura escolhida para realizar a otimização de Benchmark foi a Lista Encadeada. Para a sua otimização, foi feita uma versão da Lista Encadeada "compactada". Enquanto na lista encadeada original, cada nó é um "malloc" separado, e o sistema pode espalhá-los pela memória, além de cada malloc possuir um custo administrativo, a versão compactada aloca um único bloco grande e guarda os nós dentro desse bloco, usando índices inteiros no lugar de ponteiros.

Dentro dessa otimização, foram feitas duas versões (v_1 e v_2) de Listas Compactadas, a fim de testá-las e obter mais resultados acerca da eficiência da lista original, com sua forma compactada. É importante ressaltar, que tanto a lista original e as duas versões da lista compactada, possuem a mesmamesma complexidade, $O(n)$. Nenhuma é assintoticamente melhor, ou seja, todas percorrem a lista nó a nó. O que muda é a constante no big-O, e essa constante é relacionada pelo cache.

A lista original faz N alocações separadas, e os nós ficam espalhados pela memória, logo, percorrê-los gera muitas "faltas de cache" (idas caras à RAM). A v_1 guarda os dados num bloco contíguo, de 120 bytes, então a leitura é sequencial e o processador adivinha e pré-carrega os próximos nós. A v_2 separa o dado "quente" (só a chave do dado e o índice, possuindo 8 bytes) do dado "frio" (o registro inteiro), de modo que cabem aproximadamente 8 células por linha de cache — uma única busca à RAM traz vários nós de uma vez.

Portanto, a partir dos testes feitos com a lista original e as listas compactas v_1 e v_2 , os resultados foram surpreendentes. A compactação das versões v_1 e v_2 tornou a construção da lista mais rápida, pois com um malloc de um grande bloco no lugar de vários menores, tornou a estrutura muito mais dinâmica e eficiente, e usou menos memória. A separação quente/frio da versão v_2 recuperou e superou a busca em larga escala. Ademais, é válido ressaltar que o benefício de uma otimização de memória depende da relação entre o tamanho dos dados e o cache, e que compactuações simplórias podem piorar a busca

dentro da lista por não ser tratado o tamanho do nó. Assim, conclui-se que a compactação não altera a classe de complexidade (continua $O(n)$), mas reduz a constante ao respeitar a hierarquia de memória — e o ganho é tanto maior quanto maior o volume de dados, chegando a aproximadamente $4,4\times$ em 200 mil registros. Logo abaixo, encontra-se o gráfico e a tabela dos resultados a partir dos testes realizados a partir de 10.000 dados, que validam o que foi dito anteriormente.

Tabela 2: Resultados do Benchmark de Otimização (Nó Original: 128 bytes — Compactado: 120 bytes)

Versão	N	ins (ms)	busca (us/op)	memória (KB)
original	10000	1.77	38.9587	1250.0
compactada	10000	0.61	29.2178	1171.9
compactada v2	10000	0.59	18.9885	1210.9
original	25000	4.05	118.8711	3125.0
compactada	25000	2.84	81.3172	2929.7
compactada v2	25000	1.90	53.7537	3027.3
original	50000	5.14	415.7669	6250.0
compactada	50000	2.56	235.7004	5859.4
compactada v2	50000	2.06	126.8198	6054.7
original	100000	10.78	3300.6703	12500.0
compactada	100000	10.07	903.4258	11718.8
compactada v2	100000	6.65	325.5249	12109.4
original	200000	17.89	7221.2717	25000.0
compactada	200000	22.06	3609.8451	23437.5
compactada v2	200000	16.23	1648.5790	24218.8

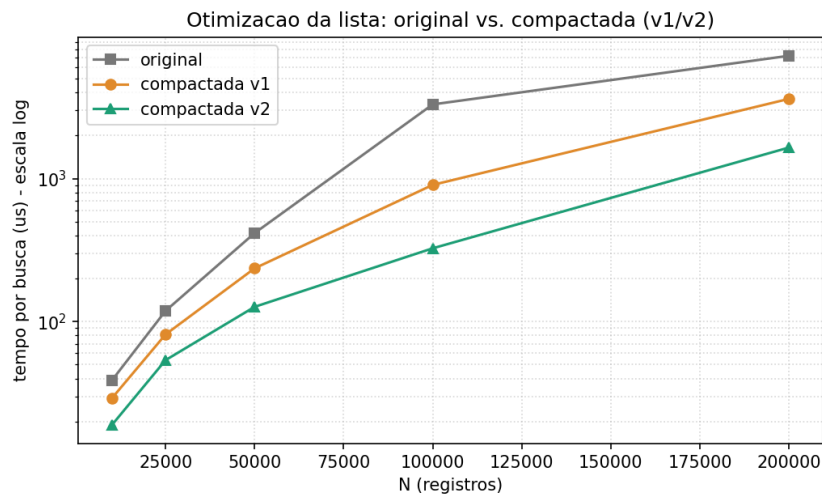


Figura 4: Gráfico do tempo por busca de memória em função do número de dados.

8 Benchmarks com Restrições - Tabela Hash e Lista

A Tabela Hash e a Lista foram a estrutura escolhidas para realizarem o benchmark com as restrições. Essa escolha foi feita, devido a grande diferença entre essas duas estruturas, tanto quanto a diferença de suas complexidades temporais.

8.1 R5 - Descarte Automático de Dados Antigos Quando a Memória Atinge um Limite Crítico.

A restrição R5 foi escolhida pois o sistema é descrito como um fluxo contínuo de jogos. E a dúvida principal desse quesito seria a consequência se os dados não pararem de chegar na Tabela Hash. A consequência seria ficar sem memória para receber mais dados posteriormente. A janela deslizante responde isso mantendo só os "n" registros mais recentes e descartando os antigos.

Foi apresentado como resultado da benchmark na R5 um fluxo de 351.473 registros e janela de 5.000 dados, a estrutura terminou com exatamente 5.000 elementos. Portanto, conclui-se que a memória não cresce com o tamanho do fluxo, mas fica fixa em $O(n)$. Isso prova que dá para processar um stream potencialmente infinito com memória limitada, simplesmente apagando os dados antigos.

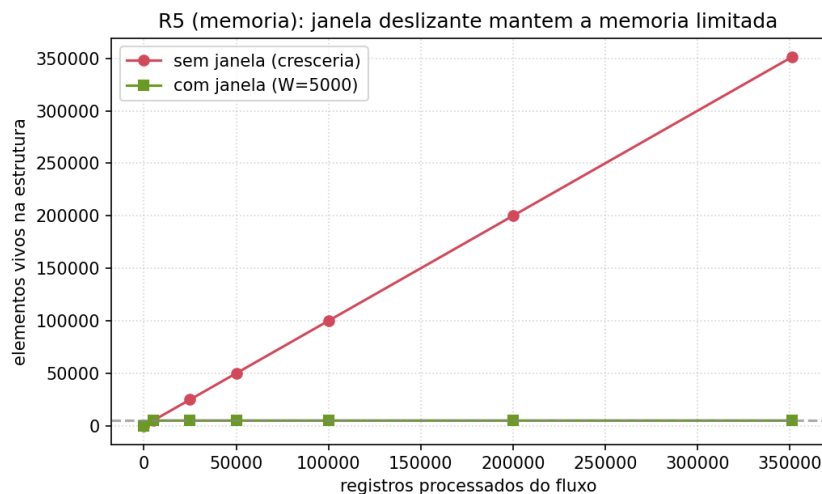


Figura 5: Gráfico do benchmark em R5.

8.2 R7 - Restrição de Orçamento Computacional

A restrição R7 foi escolhida para reformular a complexidade como um requisito de sistema de tempo real, supondo que para cada busca tivesse um prazo máximo de tempo para realizar essa operação. Aqui o contraste lista vs hash é o centro — é o $O(n)$ vs $O(1)$ visto pela ótica de "quem cumpre o prazo".

O benchmark com a R7 apresentou um resultado sob o prazo de 1 μ s, a lista estourou o limite em 100% das buscas, e a hash em apenas 0,1% das buscas feitas. Logo pode-se analisar que num sistema com requisito de tempo real, a lista seria inviável pois quase nunca entrega a tempo, enquanto a hash quase sempre cumpre. Logo o custo $O(n)$ apresenta uma falha para com essa restrição.

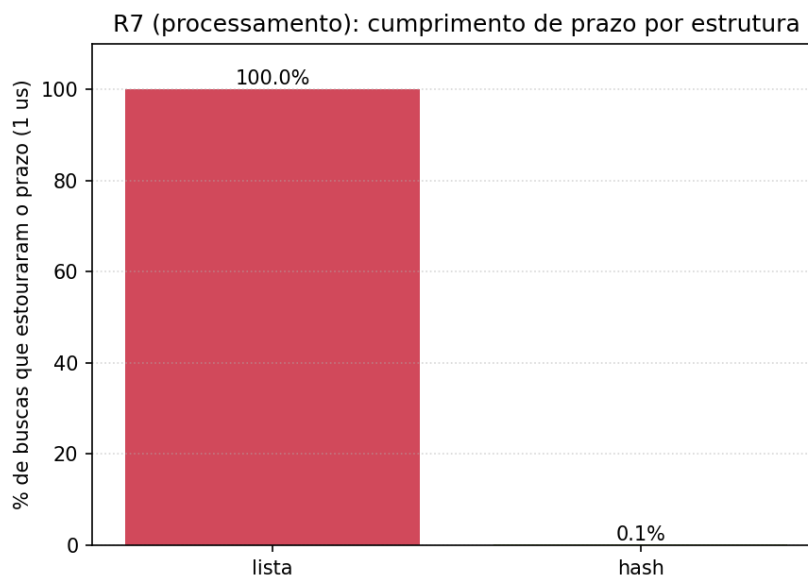


Figura 6: Gráfico do benchmark em R7.

8.3 R12 - Simulação de Alta Latência no Acesso a Banco de Dados

A restrição R12 foi escolhida para testar se o gargalo não for a estrutura, mas a chegada dos dados para a estrutura. Dessa forma, foi simulado uma fonte que demora 100 μ s por registro. De novo foi usado a Hash e a Lista, para mostrar que, sob latência alta, as duas se comportam igual.

Os resultados do benchmark com a R12 mostram que a ingestão demorou 201 ms via hash e 203 ms via lista, ambas praticamente coladas no piso teórico de 200 ms (que é a soma das latências). Dessa forma pode-se interpretar que quando a latência domina, a escolha da estrutura se torna irrelevante. As duas estruturas, tão diferentes no benchmark normal, ficaram indistinguíveis nessa restrição. Portanto, é possível concluir que não é proveitoso otimizar a estrutura se o gargalo está no I/O.

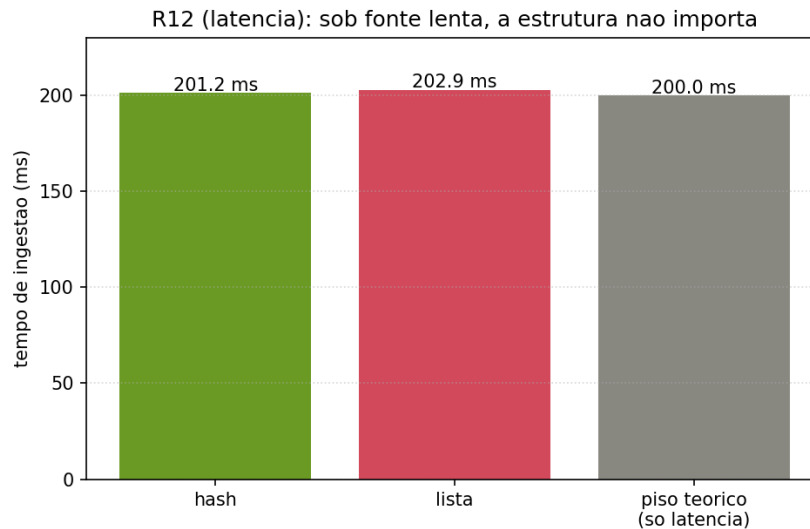


Figura 7: Gráfico do benchmark em R12.

8.4 R18 - Dados

A restrição R18 foi escolhida porque conecta diretamente com a análise de jogador MVP, operação realizada por esse projeto. Aqui não se compara as estruturas, se compara a conclusão (a média de pontos) com dados limpos contra dados corrompidos. Nessa restrição é abordado e trabalhado sobre qualidade de dado, não sobre a estrutura em si.

Os resultados da benchmark com a R18 mostram que com 10% de valores anômalos (35.422 registros adulterados), a média de pontos saltou de 10,05 para 1.016,76. Assim, pode-se interpretar que as estruturas continuaram funcionando perfeitamente; guardaram e recuperaram os dados corrompidos com a mesma eficiência de sempre. Mas a conclusão analítica, de quem é o MVP, ficou completamente distorcida. Portanto a eficiência da estrutura não protege contra dados ruins. Isso comprova e justifica a importância do tratamento de dados (o item 4 do relatório).

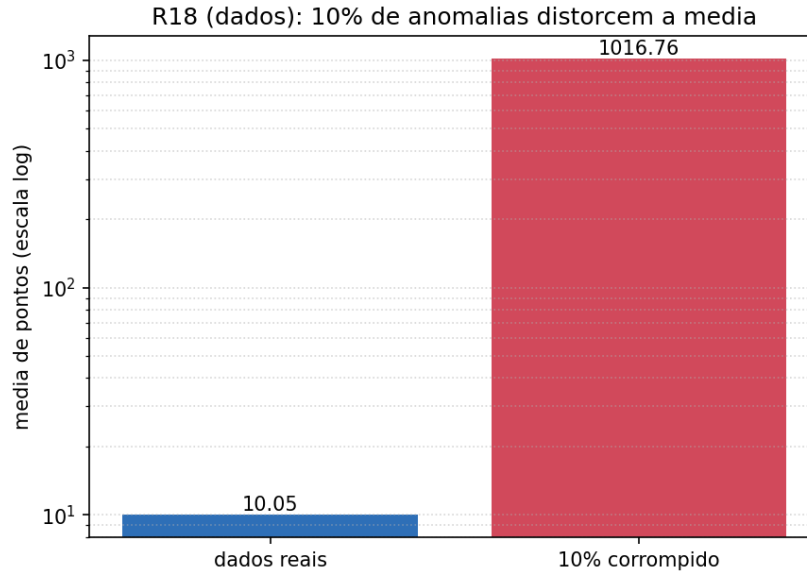


Figura 8: Gráfico do benchmark em R18.

8.5 R24 - Uso de Algoritmos Menos Otimizados

A restrição R24 foi escolhida pois o seu sistema usa ordenação na operação de estatísticas e no ranking. Nessa restrição, se houvesse uma troca, por exemplo, de um bom algoritmo (quicksort, $O(n \log n)$) por um ruim (selection sort, $O(n^2)$), mostraria o impacto de uma má decisão algorítmica. Nessa restrição não é a estrutura que é trabalhada, mas sim o algoritmo de ordenação.

Os resultados apresentados pela benchmark com a R24 mostram que o selection sort foi $551\times$ mais lento que o quicksort para ordenar 20.000 pontuações. Logo, é possível interpretar que uma decisão algorítmica ruim ($O(n^2)$ em vez de $O(n \log n)$) degrada o sistema em escala, muito mais do que qualquer restrição de hardware. Isso mostra que a escolha do algoritmo pode pesar tanto quanto a escolha da estrutura.

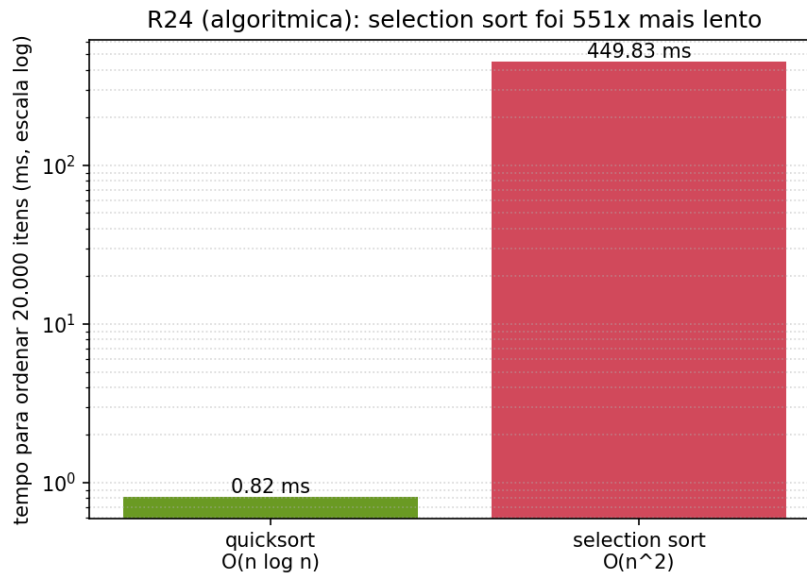


Figura 9: Gráfico do benchmark em R24.

9 Complexidade Temporal Teórica e Assintótica Real

Utilizando gráficos com as complexidades temporais e os dados obtidos nos Benchmarks, é possível comparar a complexidade temporal teórica com o tempo de real de execução de cada uma das estruturas usadas nesse projeto. Esses gráficos foram feitos com base nos dados testados pelo software desse projeto.

9.1 Lista Encadeada Simples

A lista encadeada simples, segundo a teoria, possui complexidade algorítmica $O(n)$. A curva medida acompanha a reta teórica quase sobreposta até 25 mil, e depois descola para cima, ficando mais inclinada que 1. Inclinação 1,5 no geral. Logo, isso confirma a complexidade $O(n)$ na faixa pequena, mas fica supralinear em escala devido ao efeito do cache. Enquanto a lista cabe na memória rápida do processador, ela é $O(n)$ normal. Quando ela cresce além do cache, cada nó visitado vira uma ida cara à RAM, e a constante escondida no $O(n)$ aumenta. Dessa forma, o resultado é a complexidade teórica continua sendo $O(n)$, mas a complexidade observada parece pior que linear em larga escala.

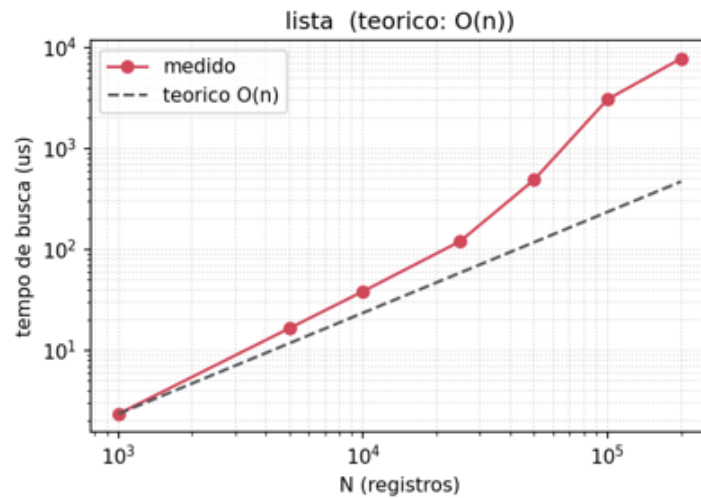


Figura 10: Gráfico da complexidade temporal da Lista Encadeada.

9.2 Árvore AVL

A Árvore AVL possui complexidade $O(\log(n))$, tanto em sua operação de busca, quanto nas operações de inserção e remoção. Além disso, a AVL continua a ter complexidade $O(\log(n))$ em seu pior caso. No gráfico, a inclinação começa pequena, quase sublinear, mas logo demonstra um pico quando N atinge 25 mil.

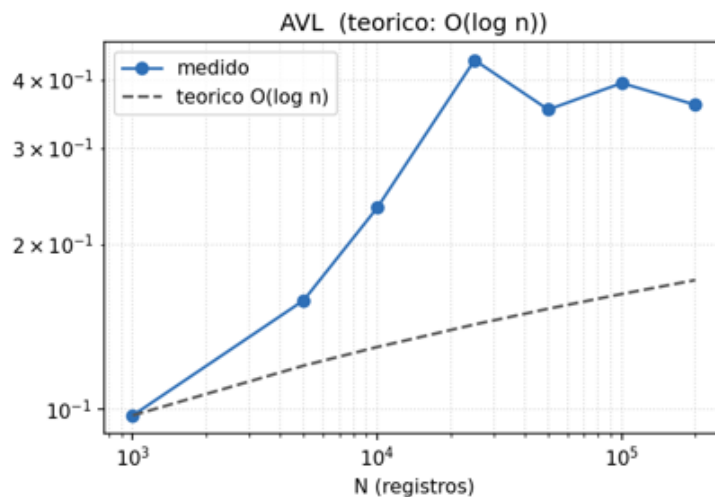


Figura 11: Gráfico da complexidade temporal da Árvore AVL.

9.3 Tabela Hash

A Tabela Hash possui complexidade $O(1)$. A curva medida é quase plana (inclinação 0,2). Ela sobe um pouco de 0,027 para 0,084 μs , mas isso não é crescimento algorítmico, mas

sim um overhead da CPU. Logo, os dados obtidos confirmam $O(1)$: o tempo praticamente não responde ao tamanho, sendo compatível com os modelos previstos pela literatura.

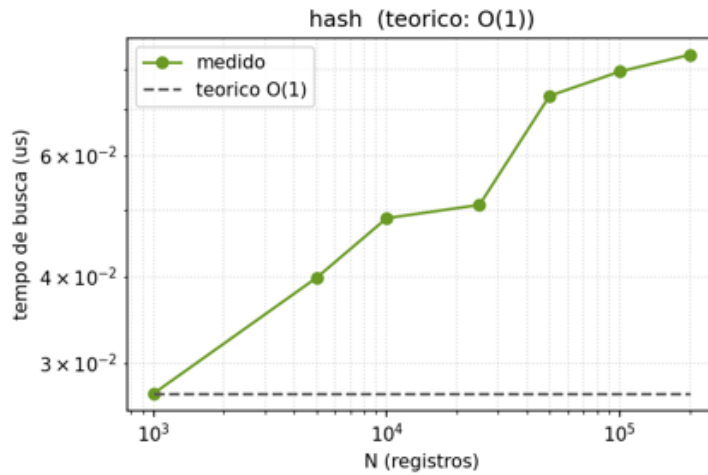


Figura 12: Gráfico da complexidade temporal da Tabela Hash.

9.4 Skip List

Assim como a Árvore AVL, a Skip List também possui complexidade $O(\log(n))$. Em 0,25 ela possui uma inclinação pequena, de forma claramente sublinear, e ela começa a crescer de forma logarítmica, como previsto pelo modelo teórico.

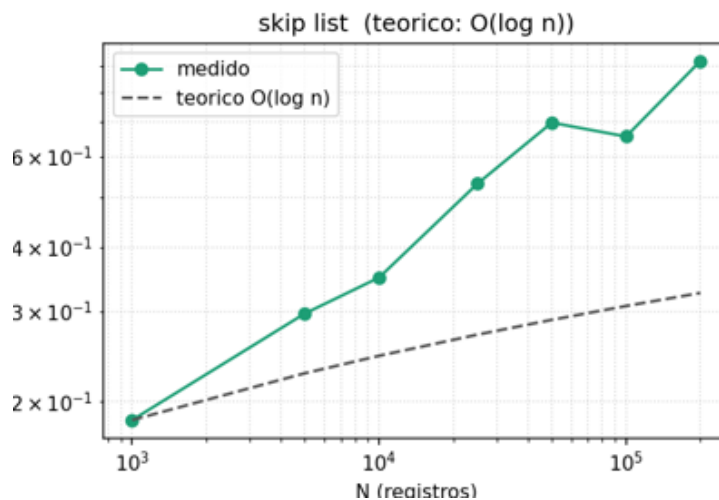


Figura 13: Gráfico da complexidade temporal da Skip List.

9.5 Bloom Filter

O Bloom Filter não tem "complexidade de busca" no mesmo sentido das outras estruturas, pois a operação dele tem complexidade $O(k)$, onde k é o número de funções hash (uma

constante, no caso deste projeto, $k=7$), independente de n . Então ele é efetivamente $O(1)$, mas por um motivo diferente da hash: não há dados para percorrer, só k bits para verificar. E ele não realiza a operação de remover, então a métrica de remoção não se aplica à essa estrutura, pois desligar bits corromperia outros elementos do Bloom Filter.

10 Conclusão

A construção das matrizes de dados em torno do dataset de basquete norte-americano permitiu uma contextualização proveitosa e imersiva ao universo de estruturas de dados probabilísticas e clássicas. As limitações observadas em cada estrutura provam ser complementares — em que a falha de lentidão temporal da busca na Lista Encadeada é suplantada pela Tabela Hash, ao mesmo passo em que o Bloom Filter atua maximizando a gestão dos blocos de pertinência com memória ínfima. A validação do conhecimento alcançado através deste simulador atende perfeitamente à métrica de avaliação solicitada, consolidando que o embasamento crítico e prático sobre a alocação e recuperação veloz da informação está alinhado às demandas e desafios dos modernos ecossistemas de software. Além de evidenciar que cada estrutura possui sua vantagem e desvantagem, e que em cenários determinados e proveitosos, todas as estruturas apresentadas nesse relatório cumprem o papel determinado a esta.

11 Uso de Inteligência Artificial

O uso de Inteligências Artificiais foi aproveitada para a assistência na codificação e implementação das estruturas de dados. Todos os gráficos e tabelas foram gerados pela IA claude.

Porém, como ordenado no escopo do projeto, o relatório não foi escrito com uso de IA.

Link da conversa com IA: <https://claude.ai/share/a6ed843c-ad3e-4059-a780-9969753e1680>

Link GitHub: <https://github.com/pedrorscoelho/Projeto-ESD/tree/main/github>